DATA VISUALIZATION

Complete Exam Preparation Guide

Microsoft Power BI • DAX Functions • Visual Best Practices

Data Types • Relationships • Canvas Views

Covers all syllabus topics with examples, scenarios & tips

■ Table of Contents

Chapter 1 — Power BI & Canvas Views

Power BI Desktop is Microsoft's free business intelligence tool that lets you connect to data, transform it, build a data model, write DAX measures, and design interactive reports — all in one application.

Three Canvas Views (Must Know for Exam)

■ Power BI has exactly THREE views accessible from the left navigation bar. Understanding each view's purpose is a common exam question.

View	Icon	What You Do Here
Report View	■	Design your dashboards and reports. Drag visuals onto the canvas. Add slicers, cards, charts. This is the default view when you open Power BI.
Data / Table View	■	See the actual data in table format. Browse rows and columns. Verify calculated columns. Create new calculated columns using DAX here.
Model / Relationships View	■	See all tables and how they are connected. Draw relationships between tables. Set cardinality (One-to-Many) and cross-filter direction.

How to Switch Between Views

Look at the **left sidebar** in Power BI Desktop. You will see three icons stacked vertically:

• Click the **chart icon** → Report View (design canvas)

• Click the **table icon** → Data View (see rows of data)

• Click the **diagram icon** → Model View (manage relationships)

Key Terms to Know

Term	Meaning
Canvas	The blank white area where you place visuals in Report View
Pane / Panel	Side panels: Visualizations pane (right), Fields pane (right), Filters pane (right)
Slicer	A visual that filters other visuals on the same page
Measure	A DAX calculation that aggregates data (SUM, COUNT, etc.)
Calculated Column	A new column added to a table using a DAX formula

Cardinality	The type of relationship: One-to-One, One-to-Many, Many-to-Many

Chapter 2 — Connecting Data & Relationships

Step 1 — Importing / Connecting Data

Power BI can connect to many data sources. For your labs and exam, the most common source is Excel (.xlsx) or CSV (.csv) files.

Step	Action	Notes
1	Click Get Data (Home ribbon)	Or use the splash screen shortcut
2	Select Excel Workbook or Text/CSV	Choose based on file type
3	Browse and select your file	Navigate to the file location
4	Check/select the table(s) to import	Preview appears on the right
5	Click Transform Data to open Power Query	For cleaning; or Load to import directly
6	In Power Query: clean, rename, change types	Critical for correct analysis
7	Click Close & Apply	Loads cleaned data into the model

Power Query — Common Cleaning Operations

■ Power Query is a data transformation engine. You access it by clicking Transform Data. All changes are recorded as steps — you can undo any step.

Operation	How To Do It	Why Important
Change Data Type	Select column → Data Type dropdown → choose Date/Number/Text	Wrong types cause calculation errors
Remove Duplicates	Select column → right-click → Remove Duplicates	Prevents double-counting
Remove Nulls/Blanks	Filter column → uncheck null/blank	Dirty data breaks measures
Trim Text	Select column → Transform → Format → Trim	Removes hidden spaces
Replace Values	Right-click column → Replace Values	Standardize categories
Split Column	Select column → Split Column → By Delimiter	Separate combined fields
Merge Columns	Select columns → Add Column → Merge Columns	Create composite keys
Rename Column	Double-click column header	Consistent naming
Filter Rows	Click dropdown arrow on column → filter	Remove invalid records

Creating Relationships (Model View)

Relationships allow Power BI to connect data from different tables. You MUST create relationships correctly for your visuals and measures to work properly.

How to Create a Relationship:

1. Switch to **Model View** (click diagram icon on left sidebar)
2. You will see all your tables as boxes with column names listed
3. **Click and drag** a column from one table to the matching column in another table
4. A line appears between the tables — this is the relationship
5. Double-click the line to configure settings (cardinality, cross-filter direction)

Relationship Types — Cardinality

Type	Symbol	Meaning	Common Example
One-to-Many (1:*)	1 → *	One row in Table A matches MANY rows in Table B	Patients → Visits (one patient, many visits)
One-to-One (1:1)	1 → 1	Each row matches exactly one row in the other table	Employee → EmployeeDetails
Many-to-Many (*:*)	* → *	Many rows on both sides — avoid if possible	Products → Orders

■ Exam Tip: For hospital lab — Patients[PatientID] → Visits[PatientID] is One-to-Many. Always set Cross-filter direction to Single unless you need bidirectional filtering.

Hospital Lab Relationships (from Lab 01)

```
Patients[PatientID] → Visits[PatientID] (One-to-Many)
Doctors[DoctorID] → Visits[DoctorID] (One-to-Many)
Departments[DeptID] → Visits[DepartmentID] (One-to-Many)
Departments[DeptID] → Doctors[DepartmentID] (One-to-Many)
```

Chapter 3 — DAX Functions Complete Reference

DAX (Data Analysis Expressions) is the formula language used in Power BI. It is used to create **Measures** (aggregations) and **Calculated Columns** (row-by-row formulas). DAX functions look similar to Excel formulas.

■ KEY DIFFERENCE: Measures are calculated on the fly based on filters/slicers. Calculated Columns are computed once and stored in the table. Use Measures for totals; use Calculated Columns for row-level logic.

Essential DAX Functions by Category

Aggregation Functions

SUM	`SUM(column)`	`SUM(Visits[BillAmount])`	Total of all values
COUNT	`COUNT(column)`	`COUNT(Visits[VisitID])`	Count of non-blank values
COUNTA	`COUNTA(column)`	`COUNTA(Patients[Name])`	Count including text
COUNTROWS	`COUNTROWS(table)`	`COUNTROWS(Visits)`	Count all rows
AVERAGE	`AVERAGE(column)`	`AVERAGE(Sales[Amount])`	Mean value
MIN	`MIN(column)`	`MIN(Orders[OrderDate])`	Smallest value
MAX	`MAX(column)`	`MAX(Orders[OrderDate])`	Largest value
DISTINCTCOUNT	`DISTINCTCOUNT(column)`	`DISTINCTCOUNT(Sales[Channel])`	Count unique values

Mathematical Functions

DIVIDE	`DIVIDE(num, denom, [alt])`	`DIVIDE([Profit],[Revenue], 0)`	Safe division (no error if 0)
SUMX	`SUMX(table, expression)`	`SUMX(Sales, Sales[Qty]*Sales[Price])`	Row-by-row sum
AVERAGEX	`AVERAGEX(table, expression)`	`AVERAGEX(Orders,Orders[Profit])`	Row-by-row average
ROUND	`ROUND(number, digits)`	`ROUND([Avg Bill], 2)`	Rounded number
ABS	`ABS(number)`	`ABS([Variance])`	Absolute value

Logical Functions

IF	`IF(condition, true, false)`	`IF([Sales]>1000,"High","Low")`	Value based on condition
SWITCH	`SWITCH(expr, val1,res1,...)`	`SWITCH([Month],1,"Jan",2,"Feb")`	Match-based result
AND	`AND(cond1, cond2)`	`AND([Age]>18,[Score]>50)`	True if both true
OR	`OR(cond1, cond2)`	`OR([Status]="A",[Status]="B")`	True if either true
NOT	`NOT(condition)`	`NOT(ISBLANK([Value]))`	Opposite of condition
ISBLANK	`ISBLANK(value)`	`ISBLANK(Patients[Phone])`	True if blank/null

Text Functions

CONCATENATE	`CONCATENATE(text1,text2)`	`CONCATENATE([First]," "&[Last])`	Combined text
&	`text1 & text2`	`[FirstName] & " " & [LastName]`	Shorter concat method
LEFT	`LEFT(text, n)`	`LEFT([OrderID], 3)`	First n characters
RIGHT	`RIGHT(text, n)`	`RIGHT([Code], 4)`	Last n characters
LEN	`LEN(text)`	`LEN([CustomerName])`	Number of characters
UPPER	`UPPER(text)`	`UPPER([Country])`	Uppercase text
FORMAT	`FORMAT(value, format)`	`FORMAT([Date],"MMM YYYY")`	Formatted as text

Date & Time Functions

TODAY	`TODAY()`	`TODAY()`	Current date
NOW	`NOW()`	`NOW()`	Current date and time
YEAR	`YEAR(date)`	`YEAR([OrderDate])`	Year number
MONTH	`MONTH(date)`	`MONTH([OrderDate])`	Month number (1-12)
DAY	`DAY(date)`	`DAY([OrderDate])`	Day number
DATE	`DATE(y,m,d)`	`DATE(2023,1,1)`	Create a date
DATEDIFF	`DATEDIFF(start,end,interval)`	`DATEDIFF([Admit],[Discharge],DAY)`	Difference between dates
EDATE	`EDATE(date, months)`	`EDATE([StartDate],6)`	Date n months away
EOMONTH	`EOMONTH(date, months)`	`EOMONTH([Date],0)`	End of month

Chapter 4 — IF / ELSE / SWITCH in DAX

IF Statement

The IF function works exactly like Excel's IF. It checks a condition and returns one value if TRUE, another if FALSE.

Basic Syntax:

```
IF( logical_condition, value_if_true, value_if_false )
```

Simple Examples:

```
-- Example 1: Flag high sales
Sales Category = IF([Total Sales] > 10000, "High", "Low")

-- Example 2: Pass or Fail
Result = IF([Score] >= 50, "Pass", "Fail")

-- Example 3: Check blank
Has Phone = IF(ISBLANK(Patients[Phone]), "No", "Yes")
```

Nested IF (IF...ELSE IF...ELSE)

When you have more than two outcomes, nest IF functions inside each other. This is equivalent to IF / ELSE IF / ELSE in programming.

```
-- Grade classification example
Grade =
IF([Marks] >= 90, "A",
IF([Marks] >= 80, "B",
IF([Marks] >= 70, "C",
IF([Marks] >= 60, "D",
"F"))))

-- Sales performance band
Performance =
IF([Sales] >= 50000, "Excellent",
IF([Sales] >= 30000, "Good",
IF([Sales] >= 15000, "Average",
"Poor")))
```

■ Tip: Nested IFs become hard to read with 4+ conditions. Use SWITCH() instead — it is cleaner and easier to maintain.

SWITCH Function

SWITCH is a cleaner alternative to nested IFs. It compares an expression against a list of values and returns the matching result.

Two Forms of SWITCH:

```
-- Form 1: Exact match
SWITCH( expression, value1, result1, value2, result2, ..., [else_result] )

-- Form 2: SWITCH(TRUE()) — for range/condition matching
SWITCH( TRUE(), condition1, result1, condition2, result2, ..., [else_result] )
```

SWITCH Examples:

```
-- Example 1: Month number to name
Month Name =
SWITCH(MONTH([Date]),
1, "January", 2, "February", 3, "March",
4, "April", 5, "May", 6, "June",
7, "July", 8, "August", 9, "September",
10, "October", 11, "November", 12, "December",
"Unknown")

-- Example 2: Region code to full name
Region Full =
SWITCH([RegionCode],
"N", "North",
"S", "South",
"E", "East",
"W", "West",
"Unknown Region")

-- Example 3: SWITCH(TRUE()) for ranges — Sales Tier
Sales Tier =
SWITCH(TRUE(),
[Total Sales] >= 100000, "Platinum",
[Total Sales] >= 50000, "Gold",
[Total Sales] >= 20000, "Silver",
"Bronze")

-- Example 4: Age Group (SWITCH TRUE)
Age Group =
SWITCH(TRUE(),
[Age] < 13, "Child",
[Age] < 18, "Teenager",
[Age] < 60, "Adult",
"Senior")
```

IF vs SWITCH — When to Use Which?

Situation	Use IF	Use SWITCH

2 outcomes (true/false)	■ Best choice	Not ideal
3–4 exact value matches	■ Gets nested	■ Best choice
Range/condition checks	■ Works fine	■ SWITCH(TRUE())
Complex logic with AND/OR	■ More flexible	Limited
Readability	Hard with 4+ levels	■ Much cleaner

Chapter 5 — Age Calculation in DAX

Calculating age from a Date of Birth (DOB) is a very common exam requirement. Power BI provides the DATEDIFF function which makes this straightforward.

Method 1 — DATEDIFF (Recommended)

```
-- Calculate Age in Years (from Lab 01)
Age = DATEDIFF(Patients[DOB], TODAY(), YEAR)

-- Breakdown:
-- Patients[DOB] = start date (date of birth column)
-- TODAY() = end date (today's date, updates automatically)
-- YEAR = interval (returns difference in whole years)
```

Method 2 — Manual Calculation

```
-- Alternative manual method
Age = INT((TODAY() - Patients[DOB]) / 365.25)

-- Note: dividing by 365.25 accounts for leap years
```

Age Group Classification

After calculating Age, classify patients/customers into groups using SWITCH or IF:

```
-- Age Group using SWITCH(TRUE())
Age Group =
SWITCH(TRUE(),
[Age] >= 0 && [Age] <= 12, "Child (0-12)",
[Age] >= 13 && [Age] <= 17, "Teenager (13-17)",
[Age] >= 18 && [Age] <= 35, "Young Adult (18-35)",
[Age] >= 36 && [Age] <= 60, "Adult (36-60)",
[Age] > 60, "Senior (60+)",
"Unknown")

-- Simpler version using just < comparisons
Age Group Simple =
SWITCH(TRUE(),
[Age] < 13, "Child",
[Age] < 18, "Teen",
[Age] < 36, "Young Adult",
[Age] < 61, "Adult",
"Senior")
```

■ Important: The Age column must be a Calculated Column (not a measure) because it works row-by-row on each patient's DOB. Go to Data View → New Column to create it.

Scenario: Hospital Patient Age Analysis

PatientID	DOB	Age (Calculated)	Age Group
P001	15/03/1990	35	Adult
P002	22/07/2010	14	Teen
P003	01/01/1955	70	Senior
P004	30/11/2018	6	Child
P005	14/08/2000	24	Young Adult

Chapter 6 — TAT & Date Difference Calculations

Turnaround Time (TAT) measures how long a process takes — from start to end. It is used in hospitals (admission to discharge), logistics (order to delivery), and customer service (ticket open to close).

DATEDIFF — The Core Function

```
DATEDIFF( start_date, end_date, interval )

Intervals available:
SECOND -- difference in seconds
MINUTE -- difference in minutes
HOUR -- difference in hours
DAY -- difference in days (most common)
WEEK -- difference in weeks
MONTH -- difference in months
QUARTER -- difference in quarters
YEAR -- difference in years
```

TAT Examples — Hospital Scenario

```
-- TAT in Days (Admission to Discharge)
TAT Days =
DATEDIFF(Visits[AdmissionDate], Visits[DischargeDate], DAY)

-- TAT in Hours
TAT Hours =
DATEDIFF(Visits[AdmissionDate], Visits[DischargeDate], HOUR)

-- Average TAT (as a Measure)
Avg TAT Days = AVERAGE(Visits[TAT Days])

-- TAT Category (Short / Normal / Long)
TAT Category =
SWITCH(TRUE(),
[TAT Days] <= 1, "Same Day",
[TAT Days] <= 3, "Short Stay",
[TAT Days] <= 7, "Normal Stay",
"Long Stay")
```

TAT — E-commerce Scenario (Order to Delivery)

```
-- Delivery TAT in Days
Delivery Days =
DATEDIFF(Orders[OrderDate], Orders[DeliveryDate], DAY)

-- On-Time Delivery Flag
Delivery Status =
IF([Delivery Days] <= 3, "On Time", "Delayed")

-- Count of Delayed Orders (Measure)
Delayed Orders =
CALCULATE(COUNTROWS(Orders), Orders[Delivery Status] = "Delayed")
```

Date Arithmetic — Adding/Subtracting Dates

```
-- Add days to a date
Expected Delivery = Orders[OrderDate] + 3

-- Subtract dates directly (returns number of days)
Days Diff = Orders[DeliveryDate] - Orders[OrderDate]

-- Check if order is overdue
Is Overdue = IF(TODAY() > Orders[ExpectedDate], "Overdue", "On Track")

-- Days until deadline
Days Remaining = DATEDIFF(TODAY(), Orders[DueDate], DAY)
```

■■ Exam Tip: DATEDIFF always goes start → end. If end is before start, you get a negative number. Make sure your column order is correct!

Chapter 7 — Date Conversions in DAX

Date conversion means extracting parts of a date (year, month, day) or changing how a date is displayed. This is essential for grouping data by month, quarter, or year in your charts.

Extracting Date Parts

```
-- Extract Year
Year = YEAR(Sales[OrderDate]) -- Returns: 2023

-- Extract Month Number
Month Num = MONTH(Sales[OrderDate]) -- Returns: 1 to 12

-- Extract Day
Day Num = DAY(Sales[OrderDate]) -- Returns: 1 to 31

-- Extract Quarter
Quarter = QUARTER(Sales[OrderDate]) -- Returns: 1 to 4

-- Day of Week (1=Sunday, 7=Saturday)
Day of Week = WEEKDAY(Sales[OrderDate])

-- Week number of the year
Week Num = WEEKNUM(Sales[OrderDate])
```

Formatting Dates as Text

```
-- Month Name (January, February...)
Month Name = FORMAT(Sales[OrderDate], "MMMM")

-- Short Month (Jan, Feb...)
Short Month = FORMAT(Sales[OrderDate], "MMM")

-- Month-Year label for charts (Jan 2023)
Month Year = FORMAT(Sales[OrderDate], "MMM YYYY")

-- Full date as text
Date Text = FORMAT(Sales[OrderDate], "DD/MM/YYYY")

-- Quarter label
Quarter Label = "Q" & QUARTER(Sales[OrderDate]) & " " & YEAR(Sales[OrderDate])
-- Returns: Q1 2023
```

Creating a Date Table (Calendar Table)

A Date Table is a special table with one row per calendar day. It allows Power BI to do time intelligence calculations (YTD, MoM, etc.). You MUST mark it as a Date Table.

```
-- Step 1: Create the Date Table
DateTable = CALENDAR(DATE(2023,1,1), DATE(2023,12,31))

-- Or use CALENDARAUTO() to auto-detect date range from your data
DateTable = CALENDARAUTO()

-- Step 2: Add columns to the Date Table
Year = YEAR([Date])
Month Num = MONTH([Date])
Month Name = FORMAT([Date], "MMMM")
Quarter = "Q" & QUARTER([Date])
Month Year = FORMAT([Date], "MMM YYYY")
Day Name = FORMAT([Date], "dddd")

-- Step 3: Right-click the DateTable in Data view
-- → Mark as Date Table → Select the Date column
```

Month Sorting Fix

■■ Problem: When using Month Name in charts, Power BI sorts alphabetically (April, August...) instead of chronologically. Fix: In Data View → select Month Name column → Column Tools → Sort by Column → choose Month Num (1-12).

Chapter 8 — Creating Visuals in Power BI

Visuals are the charts and tables you place on the Report Canvas. Power BI has dozens of visual types. Knowing which visual to use for which data situation is a key exam skill.

Chart Selection Guide

Chart Type	Best Used For	Axis / Fields Setup	Avoid When
Bar Chart (Horizontal)	Comparing values across categories (Countries, Products)	Y-Axis: Category X-Axis: Value (SUM)	Too many categories (>15)
Column Chart (Vertical)	Same as bar, also good for time-based categories	X-Axis: Category Y-Axis: Value (SUM)	Very long category names
Line Chart	Showing trends over TIME	X-Axis: Date/Time field Y-Axis: Measure	Non-time categories on X-axis
Area Chart	Same as line but emphasizes volume	X-Axis: Date Y-Axis: Measure	Comparing many series
Pie Chart	Showing part-to-whole proportions	Legend: Category Values: Measure	More than 5-6 categories
Donut Chart	Same as pie, slightly better readability	Legend: Category Values: Measure	More than 5-6 slices
Scatter Plot	Relationship between 2 numeric values Outlier detection	X: Numeric Y: Numeric Details: Category	Categorical data
Waterfall Chart	Showing increases/decreases to a total	Category + Values	Complex multi-series data
Card	Single KPI number display	Fields: One measure	Multiple related values
Table	Detailed row-level data	Columns: Any fields	When trend matters more
Matrix	Pivot-table style: rows × columns	Rows, Columns, Values	Simple flat lists
Stacked Bar/Column	Composition within comparison	Axis+Legend+Values	Hard to compare middle segments

Step-by-Step: How to Add a Visual

```
1. Click on the Report View (chart icon on left sidebar)
2. Click anywhere on the blank canvas to deselect everything
3. In the Visualizations pane (right side), click the visual icon you want
→ A blank placeholder visual appears on the canvas
4. In the Fields pane (right side), find your table and columns
5. Drag fields to the correct wells in Visualizations pane:
→ Axis / X-Axis / Rows: Drag category or date field
→ Values / Y-Axis: Drag numeric field (auto-aggregates with SUM)
→ Legend: Drag field to color-code bars/lines by category
6. Resize the visual by dragging its corners
7. Format the visual: Click Format icon (paint roller) in Visualizations pane
→ Change colors, add data labels, modify title, adjust axis
```

Lab Visual Assignments Summary

Lab	Visual Type	Axis / X	Values / Y	Extra
Lab 01 - Hospital	Column Chart	Department Name	Total Visits	—
Lab 01 - Hospital	Bar Chart	Doctor Name	Total Billing	Slicer: Dept
Lab 01 - Hospital	Line Chart	Visit Date	Total Visits	—
Lab 01 - Hospital	Card	—	Total Patients / Visits / Billing	—
Lab 02 - Sales	Column Chart	Region	Total Net Revenue	Sort Desc
Lab 02 - Sales	Bar Chart	Channel	Total Net Revenue	—
Lab 02 - Sales	Table	CustomerName	Total Net Revenue	Top 5 filter
Lab 02 - Sales	Card	—	Returned Orders Count	—
Lab 02 - Sales	Matrix	CustomerName	Channel Count	—
Lab 06 - TechMart	Bar Chart	Country	Total Sales	—
Lab 06 - TechMart	Line Chart	Month	Sales	—
Lab 06 - TechMart	Pie Chart	Category	Sales	—
Lab 06 - TechMart	Scatter Plot	Shipments	Sales	Details: Salesperson
Lab 06 - TechMart	Waterfall	Product	Jan vs Jun Sales	—

Chapter 9 — Ugly, Bad & Wrong Visuals

Data visualization is part art and part science. A visualization must **accurately convey the data** AND be **aesthetically pleasing**. The three categories of problematic figures are:

Category	Definition	Examples
■ UGLY	Has **aesthetic problems** but is otherwise clear and informative. Data is correct, just looks bad.	• Too-bright clashing colors • Mixed fonts (3 different fonts) • Overly prominent background grid • Poor color choices (neon colors)
■■ BAD	Has **perception problems** — unclear, confusing, overly complicated, or potentially deceiving.	• Each bar has its own Y-axis (misleads comparison) • Too many categories in a pie chart • Truncated Y-axis (exaggerates differences) • 3D charts (distort proportions)
■ WRONG	Has **mathematical problems** — objectively incorrect data representation.	• No Y-axis labels (values cannot be determined) • Pie chart slices don't add to 100% • Incorrect axis scale • Using Line Chart with non-time categorical axis

Common Visualization Mistakes (Exam Questions)

Mistake	Why It Is Wrong	Correct Solution
Line chart with Product on X-axis	Products have no time order — line implies a trend/sequence that doesn't exist	Use Bar or Column Chart for categorical data
Pie chart with 10+ categories	Too many slices — impossible to compare accurately. Humans can't distinguish small angles	Use Bar Chart or limit to top 5-6 with 'Other'
Separate Y-axis for each bar	Viewers cannot compare bar heights accurately across different scales	Use one shared Y-axis for all bars
No Y-axis labels	Cannot determine actual values from bar heights	Always include a Y-axis with clear labels
Truncated Y-axis starting at 500 (not 0)	Small differences look huge — misleads the viewer	Start Y-axis at 0 for bar/column charts
KPI card with no context	A number like '■ 45,000' means nothing without comparison	Add target, prior period, or % change
3D pie/bar charts	3D perspective distorts sizes — background slices look smaller	Use flat 2D charts always

Too many colors	No clear meaning to colors, creates visual noise	Use consistent color scheme; highlight only key data

Chapter 10 — Structured, Unstructured & Semi-structured Data

Data is the raw facts. Information is what you get when data is given context and meaning. Understanding how data is organized helps you choose the right tools and storage systems.

Type	Definition	Examples	Power BI Compatible?
■ STRUCTURED	Organized in rows and columns with a defined schema. Stored in relational databases. Most accessible for BI tools.	• Excel spreadsheets • SQL Server, MySQL, PostgreSQL • Microsoft Access • Azure SQL, IBM DB2 • Google Sheets • CSV files	■ Yes — primary data source
■ UNSTRUCTURED	Ambiguous — no consistent format, rhyme, or reason. The data is unique and non-replicable in its raw form.	• Photos / Images • Videos and audio files • Text documents (Word, PDF) • Social media posts • Emails (body text) • Sensor/IoT raw streams	■■ Limited — needs preprocessing
■■ SEMI-STRUCTURED	Has some structure via tags/markers but not stored in relational tables. No fixed schema. Uses NoSQL/nonrelational systems.	• JSON files • XML files • HTML pages • NoSQL databases (MongoDB) • Email headers (metadata) • CSV with nested data	■ Yes — Power Query can parse JSON/XML

Key Differences Summary

Feature	Structured	Unstructured	Semi-structured
Format	Fixed rows & columns	No fixed format	Tags/markers present
Storage	Relational DB (SQL)	File systems, blob storage	NoSQL (MongoDB, CouchDB)
Schema	Pre-defined schema	No schema	Flexible/self-describing
Searchability	Easily searchable	Hard to search	Searchable via tags
BI Tools	Directly usable	Requires transformation	Can be parsed & loaded
Examples	Excel, SQL tables	Photos, videos, audio	JSON, XML, HTML

Data Size	Smaller, well-organized	Largest (80% of world data)	Medium

Chapter 11 — Assignment Formulas Quick Reference

All DAX formulas used across the lab assignments in one place.

Lab 01 — Hospital Dashboard

```
-- Calculated Columns
Full Name = Patients[FirstName] & " " & Patients[LastName]
Age = DATEDIFF(Patients[DOB], TODAY(), YEAR)

-- Measures
Total Billing = SUM(Visits[BillAmount])
Total Visits = COUNT(Visits[VisitID])
Total Patients = COUNT(Patients[PatientID])
Average Bill = DIVIDE([Total Billing], [Total Visits])
```

Lab 02 — E-Commerce Sales Cleaning

```
-- Measures
Total Net Revenue =
SUMX(All_Sales,
All_Sales[Quantity] * All_Sales[UnitPrice] * (1 - All_Sales[DiscountRate]))

Channel Count = DISTINCTCOUNT(All_Sales[Channel])

Returned Orders Count =
CALCULATE(COUNT(All_Sales[OrderID]), All_Sales[Status] = "Returned")
```

Lab 05 — Time Intelligence (DAX)

```
-- Base Measures
Total Sales = SUM(FactSales[SalesAmount])
Total Gross Profit = SUM(FactSales[GrossProfit])
Total Units = SUM(FactSales[Quantity])

-- YTD / QTD / MTD
Sales YTD = TOTALYTD([Total Sales], Date[Date])
Sales QTD = TOTALQTD([Total Sales], Date[Date])
Sales MTD = TOTALMTD([Total Sales], Date[Date])

-- Alternate using CALCULATE
Sales YTD2 = CALCULATE([Total Sales], DATESYTD(Date[Date]))

-- Previous Period
Sales LY = CALCULATE([Total Sales], SAMEPERIODLASTYEAR(Date[Date]))
Sales PM = CALCULATE([Total Sales], PREVIOUSMONTH(Date[Date]))

-- YoY
YoY Growth = [Total Sales] - [Sales LY]
YoY % = DIVIDE([YoY Growth], [Sales LY], 0)

-- Rolling 12 Months
Rolling 12M Sales =
CALCULATE([Total Sales],
DATESINPERIOD(Date[Date], LASTDATE(Date[Date]), -12, MONTH))

-- Fiscal YTD (April fiscal year start)
Fiscal YTD Sales = TOTALYTD([Total Sales], Date[Date], "03/31")

-- Budget
Budget Sales = SUM(BudgetMonthly[BudgetAmount])
Sales Variance = [Total Sales] - [Budget Sales]
Sales Variance% = DIVIDE([Sales Variance], [Budget Sales], 0)
```

Lab 06 — TechMart Dashboard

```
-- Date Table
DateTable = CALENDAR(DATE(2023,1,1), DATE(2023,12,31))
Year = YEAR(DateTable[Date])
Month Num = MONTH(DateTable[Date])
Month Name = FORMAT(DateTable[Date], "MMMM")
Quarter = "Q" & QUARTER(DateTable[Date])

-- Measures
Total Sales = SUM(data[SalesAmount])
Total Profit = SUM(data[Profit])
Total Shipments = SUM(data[Shipments])
```

Chapter 12 — Exam Tips & Final Checklist

Top 10 Things to Remember for the Exam

#	Tip	Details
1	Three Canvas Views	Report View (design), Data View (see rows), Model View (relationships). Know each one!
2	IF vs SWITCH	IF for 2 outcomes; SWITCH for 3+ exact matches; SWITCH(TRUE()) for ranges
3	Age Calculation	DATEDIFF(DOB, TODAY(), YEAR) — start is DOB, end is TODAY()
4	DATEDIFF Order	Always: DATEDIFF(start_date, end_date, interval) — negative if dates are wrong way
5	Line Chart Rule	X-axis MUST be a time/date field. Product on X-axis of line chart = WRONG
6	Pie Chart Rule	Maximum 5-6 slices. More than that = BAD visualization
7	Relationships	One-to-Many is most common. Drag from the 'one' side (primary key) to the 'many' side
8	Structured Data	Excel, SQL, CSV = Structured. Photos/Videos = Unstructured. JSON/XML = Semi-structured
9	FORMAT function	Use FORMAT([Date], "MMMM") for month name; "MMM YYYY" for chart labels
10	Mark Date Table	After creating DateTable with CALENDAR(), you MUST mark it as Date Table for time intelligence

Quick Exam Checklist ■

- ■ I know the 3 Power BI views and what each one is used for
- ■ I can import Excel/CSV data into Power BI
- ■ I know how to clean data in Power Query (change types, remove nulls, trim, replace)
- ■ I can create relationships between tables in Model View
- ■ I understand One-to-Many cardinality
- ■ I can write an IF statement with nested conditions
- ■ I can write a SWITCH statement (both forms)
- ■ I can calculate Age using DATEDIFF(DOB, TODAY(), YEAR)
- ■ I can calculate TAT/date differences using DATEDIFF
- ■ I know the DATEDIFF intervals: SECOND, MINUTE, HOUR, DAY, MONTH, YEAR
- ■ I can extract Year, Month, Day from a date
- ■ I can format dates using FORMAT([Date], 'format string')

■ I can create a Date Table using CALENDAR()

■ I know which chart to use for comparison, trend, composition, relationship

■ I can identify Ugly, Bad, and Wrong visualizations

■ I know the difference between Structured, Unstructured, Semi-structured data

■ I can write SUM, COUNT, AVERAGE, DIVIDE measures

■ I can write SUMX for row-by-row calculations

■ I know CALCULATE() changes the filter context

■ I have practiced all lab formulas

Best of luck in your exam! ■

You have covered all the topics: DAX functions, IF/SWITCH, Age calculations, TAT/Date differences, Date conversions, Canvas views, Relationships, Visual creation, Ugly/Bad/Wrong figures, and Data types. Practice the formulas and you will do great!